QFAN Handbuch v1.7

Feldordner, Host-Server, Client-Browser, Vault mit Rechten, Träger, Export

1. Aufbau

Grundlage ist dein Paket QFAN-Network v1.2 (Core mit EQ1-EQ5, Q-DNS, Field-DHCP, QIP, Substrate, Burst, Takt-Sync). Diese Module sind unverändert. Neu ist die Engine, die Host und Client als eigene Rollen im Hub-and-Spoke-Betrieb führt: Der Host (Server · Nexus, cyan) vergibt Field-DHCP-Slots, hält die Presence, leitet Nachrichten weiter, verwaltet den Vault und führt das WORM-Ledger. Clients (Client Browser, orange) docken an den Host an.

2. Träger (Schicht 1): der Feldordner

Standard ist das file-Substrat: ein gemeinsamer oder synchronisierter Ordner (NAS, SMB, NFS, Syncthing, Nextcloud, OneDrive) mit atomaren Slot-Dateien <ziel>.<ts_ns>.slot. Es werden keine IP-Adressen, keine Ports und kein localhost benötigt. Host und Clients wählen beim ersten Start denselben Feldordner; auf verschiedenen Rechnern oder Netzen übernimmt das Synchronisationsprogramm den Transport. Der blinde UDP-Träger (--hub, --hub-listen) ist optional und standardmäßig aus.

3. Start

Rolle	Befehl
Host mit Oberfläche	python -m qfan --role host (host\START_HOST.bat)
Host als Server	python -m qfan --role host --headless --path <Feldordner>
Client	python -m qfan --role client --name jan --path <Feldordner> (client\START_CLIENT.bat)
.exe bauen	build_exe.bat -> dist\QFAN-Host.exe, dist\QFAN-Client.exe

Q-MAC / QIP-Adressierung (Frame v4)

Jede Signatur wird über SHA-256 in eine feste Adresse QIP-XXXX-HASH06 übersetzt (qfan.de/jan = QIP-9661-6A0AC0, identisch mit resolve_qip des Chat Translator 2). Jeder Frame trägt sender_qip und target_qip direkt im Frame, unabhängig vom Substrat. Der Empfänger prüft Frame v4, Absender-Q-MAC und Ziel (eigene Q-MAC oder Broadcast QIP-0000-000000); sonst Verwerfen als QMAC_MISMATCH. Weitergeleitete Nachrichten tragen origin_qip. Öffentliche QNAT-Adressen: QIP-1400 ff.

Schichten: QTCP und QNAT

Jeder Frame läuft: Name > Q-DNS > Qubit > Ronmetrik > EQ1-EQ5 > Liouville > Field-DHCP (NODE-7xx) > QNAT (QIP-14xx) > QTCP-Burst am Taktanker 98,112 Hz > Substrat. **QTCP**: Steuer-Frames gehen sofort, Beat, Chat und Chunks im Burst im Slot-Abstand; der Tick (10 Slots, ca. 0,1 s) ist die Zeitbasis. **QNAT**: öffentliche QIP (QIP-1400 ff.) aus einem Pool (Standard 32), jeder gültige Frame ist ein Resonanz-Keep-Alive, nach --tick-timeout Ticks (Standard 300) ohne Resonanz oder bei dauerhafter Torsion folgt die Eviction, bei vollem Pool wird der Handshake abgelehnt.

4. Ablauf und Oberfläche

- **Login:** Der Client sendet einen Handshake. Der Host prüft das Qubit (Liouville, EQ1-EQ5), vergibt NODE-7xx und QIP. Bei allen anderen poppt „■ qfan.de/jan ist jetzt online“ auf. Beat im Burst; ohne Resonanz folgt die QNAT-Eviction, Clients verbinden sich nach Host-Ausfall automatisch neu.
- **Control Center:** Knotenliste mit Slot, Chat an alle oder privat (läuft über den Host).
- **Monitor:** Scope für den Träger (98,112 Hz grün, Glättung gelb, Interlock rot bei 196,224 Hz), Residuum, Auto-Logs, Telemetrie.
- **Träger-Hopping:** Residuum über 0,25 schaltet auf 196,224 Hz, nach drei ruhigen Frames (unter 0,08) zurück auf 98,112 Hz. Je Hop ein Auto-Log und ein Ledger-Eintrag.
- **QNAT-Eviction:** Knoten ohne Resonanz verlieren nach 300 Ticks QIP und NODE-Slot; freie Adressen werden ohne Duplikate wieder vergeben.

- **Gate:** Jeder Frame wird geprüft (Liouville, EQ1-EQ5, Residuum gegen den Q-DNS-Vektor des Absenders; über 0,25 = TORSION_ESCAPE).

Quantum Vault mit Rechten

- Der Host legt Dateien ab; sie sind zunächst nur für ihn lesbar. Rechte erteilt er pro Client oder für alle (Rechtsklick oder Menü QVault), er kann sie entziehen.
- Clients sehen und rufen nur Freigegebenes ab (SHA-256-geprüft). Nicht berechtigte Abrufe werden abgelehnt und im Ledger vermerkt.
- Clients injizieren Dateien ins Feld (INJECT_<client>_<zeit>_<name>), lesbar für Host und Besitzer. Quota 5 GB, einzelne Datei bis 64 MB, Namen bereinigt, nichts wird überschrieben.
- Menüs: Host: Datei ablegen, Import Felddaten (CSV), Analyse, QVault. Client: Härtetest (künstliche Torsion), Datei injizieren, Abrufen.

OQ-DHC V13 Testbed

Core, Chat Translator 2 und RCS-Stresstest liegen unverändert unter oqdhc/. Anbindung: Menü Ansicht > RCS-Stresstest, Headless /rcs [qubits] [zyklen] [rauschen] und /qip <name>. Jeder Lauf erscheint als RCS_RUN im WORM-Ledger. Die in der Testbed-README genannte libron_avx512.c war nicht im Upload.

5. Ledger und Telemetrie

- WORM-Ledger: SHA-256-Kette, als JSONL gespeichert, beim Start geprüft. Menü Ansicht > WORM-Ledger prüfen. Erfasst u. a.: CLIENT_LOCKED, CLIENT_LEFT, LEASE_EVICTED, MSG, FILE_SHARED, ACL_GRANT/REVOKE/DENIED, FILE_INJECTED, CARRIER_HOP_UP/DOWN, Gate-Rejects.
- Telemetrie: Ringpuffer (5000 Frames) mit RON_SMOOTHING, TORSION_ESCAPE, LIOUVILLE_REJECT, EQ_MISMATCH, Residuum, det(J). Export als .log und als PDF über das Menü Datei (Report mit Knoten, Träger, Telemetrie, Ledger, Vault und Feld-Stream).
- Stress-Simulation: im Headless-Modus /stress 1000 schickt synthetische Frames mit Rauschen durch den echten Core und zeigt die Auswertung.

6. Headless-Befehle

```
/say <text>   /to <name> <text>   /who   /files   /ledger   /stats   /stress [n]   /quit
Host:   /share <datei>  /grant <datei> <name|*>  /revoke <datei> <name|*>  /import <csv>  /expor
Client: /get <datei> [ziel]  /inject <datei>  /torsion  /export <ziel.log|.pdf>
Text ohne / geht an alle.
```

7. Tests und Stand

- 52 automatische Tests (python -m unittest tests.test_engine tests.test_ui_smoke tests.test_oqdhc tests.test_qtcp_qnat tests.test_qmac): Login und Presence, Timeout, Neuverbindung, Chat, Vault-Abruf, Rechte erteilen/entziehen/ablehnen, Client-Injektion, Quota, Q-MAC (Frame v4, Fälschungen), QTCP-Burst und Tick, QNAT-Lease, Overflow und Eviction, Träger-Hopping, QWI-Frame, CSV-Analyse, Log- und PDF-Export, Ledger-Manipulation, Stress-Simulation und eine Prüfung, dass keine localhost-Adressen in den Quellen stehen.
- Host und Client liefen zusätzlich als zwei getrennte Prozesse nur über den Feldordner (Login, Chat, Injektion, Härtetest, Abmeldung), ohne Socket.
- Die Fenster selbst (tkinter) wurden hier nur mit simuliertem tkinter getestet; bitte einmal real öffnen. Ein Test über zwei echte Rechner mit synchronisiertem Ordner steht aus.
- Nachrichten sind unverschlüsselt.